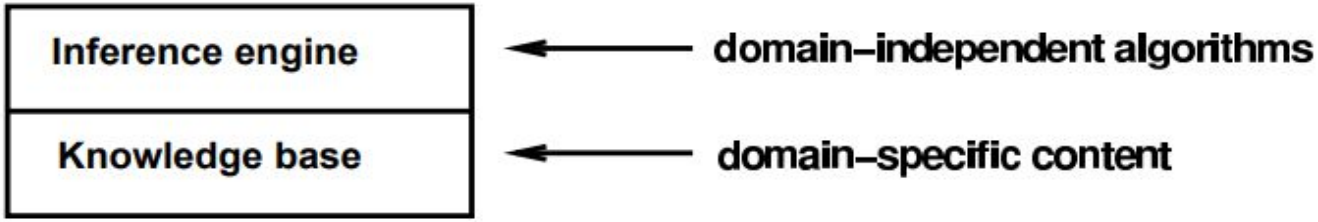
Logical agents

Slide- Stuart Russell and Peter Norvig

Outline

- ♢ Knowledge bases
- ♢ Wumpus world
- ♢ Logic in general
- ♢ Propositional (Boolean) logic
- ♢ Normal forms
- ♢ Inference rules

Knowledge bases



Knowledge base = set of sentences in a formal language

Declarative approach to building an agent (or other system):
TELL it what it needs to know

Then it can ASK itself what to do—answers should follow from the KB

Agents can be viewed at the knowledge level
i.e., what they know, regardless of how implemented

Or at the implementation level
i.e., data structures in KB and algorithms that manipulate them

A simple knowledge-based agent

function KB-AGENT(*percept*) **returns** an *action*
  static: *KB*, a knowledge base
    t, a counter, initially 0, indicating time

  TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))
  action $\leftarrow$ ASK(*KB*, MAKE-ACTION-QUERY(*t*))
  TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))
  $t \leftarrow t + 1$
  return *action*

The agent must be able to:

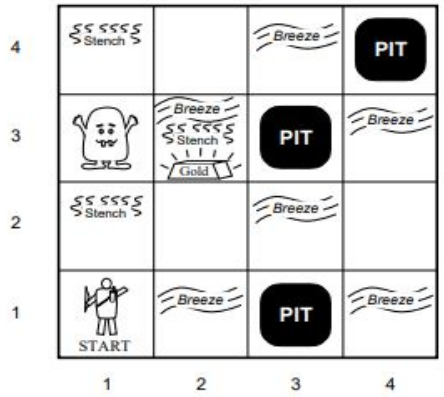
- Represent states, actions, etc.
- Incorporate new percepts
- Update internal representations of the world
- Deduce hidden properties of the world
- Deduce appropriate actions

Wumpus World PAGE description

Percepts Breeze, Glitter, Smell

Actions Left turn, Right turn,
Forward, Grab, Release, Shoot

Goals Get gold back to start
without entering pit or wumpus square



Environment

- Squares adjacent to wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter if and only if gold is in the same square
- Shooting kills the wumpus if you are facing it
- Shooting uses up the only arrow
- Grabbing picks up the gold if in the same square
- Releasing drops the gold in the same square

Wumpus world characterization

Is the world deterministic??

Is the world fully accessible??

Is the world static??

Is the world discrete??

Wumpus world characterization

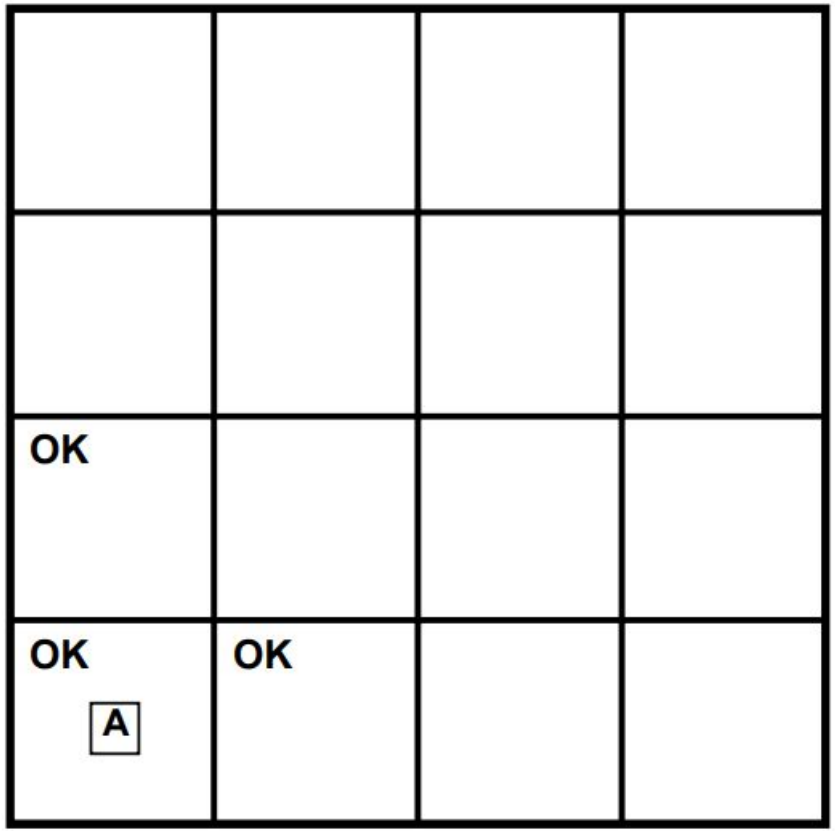
Is the world deterministic?? Yes—outcomes exactly specified

Is the world fully accessible?? No—only local perception

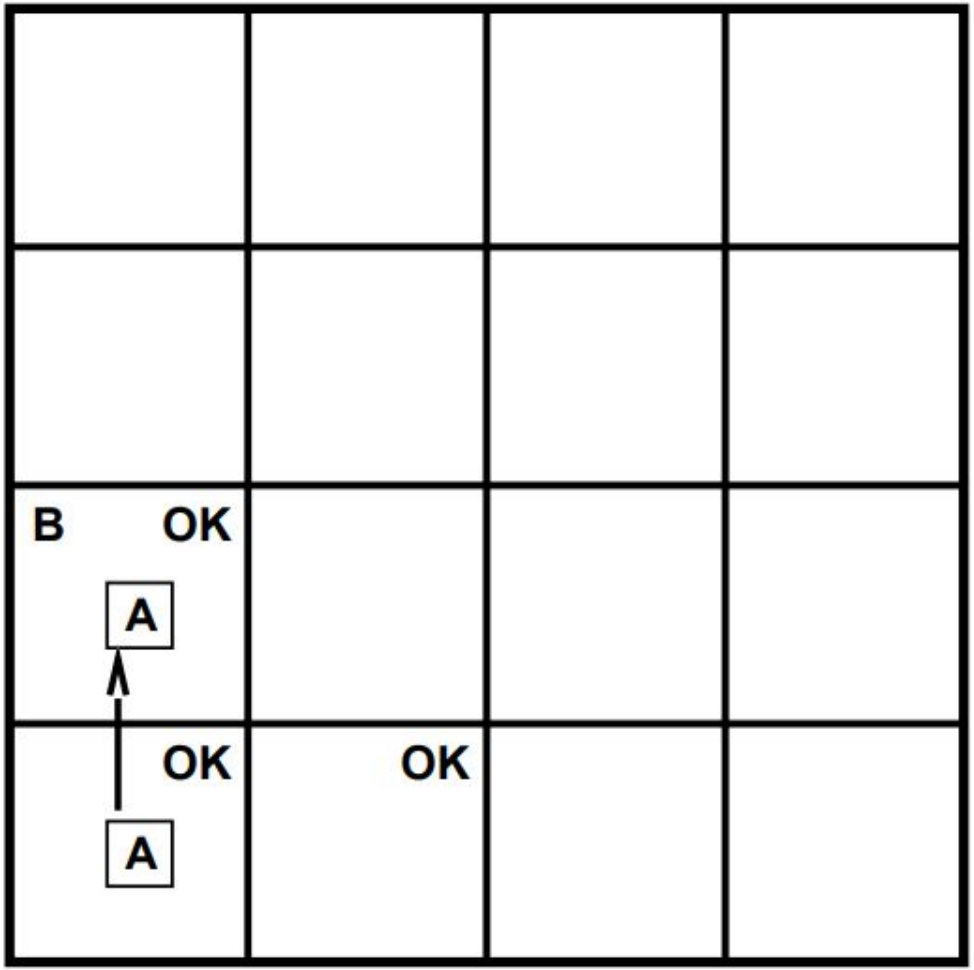
Is the world static?? Yes—Wumpus and Pits do not move

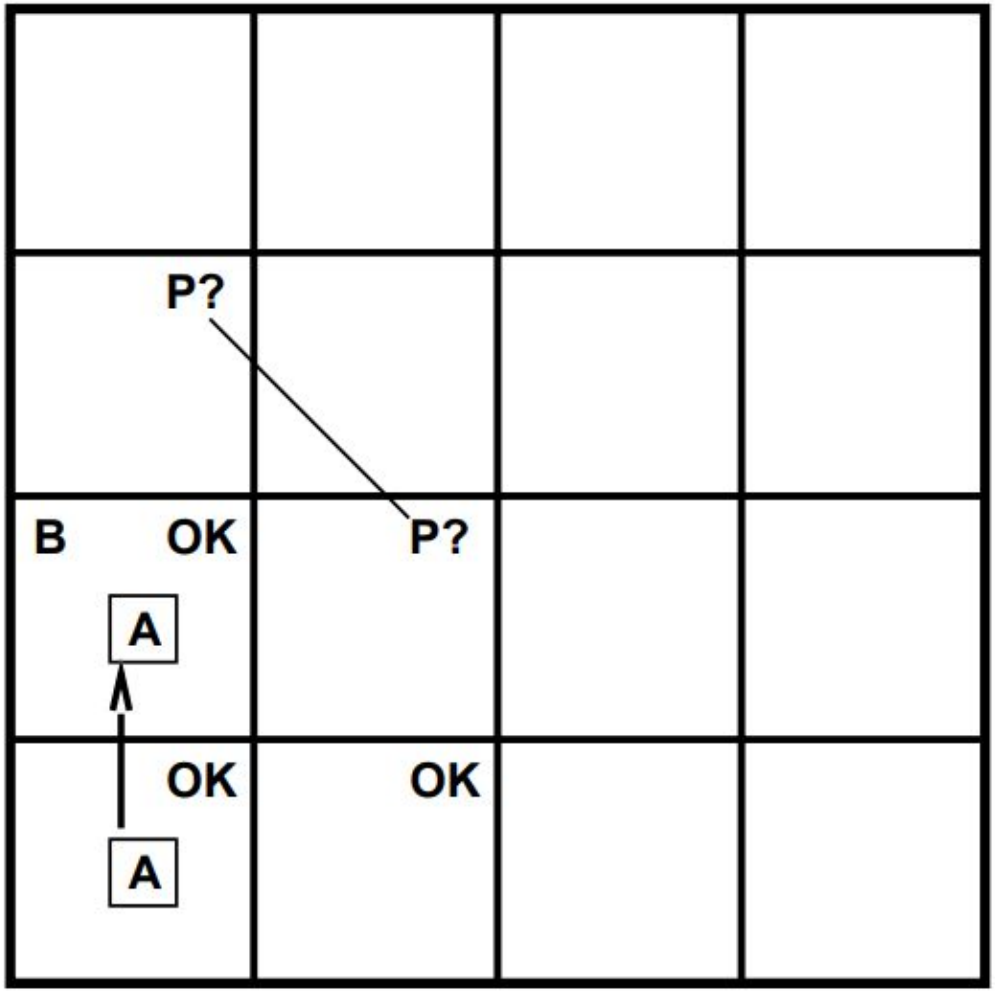
Is the world discrete?? Yes

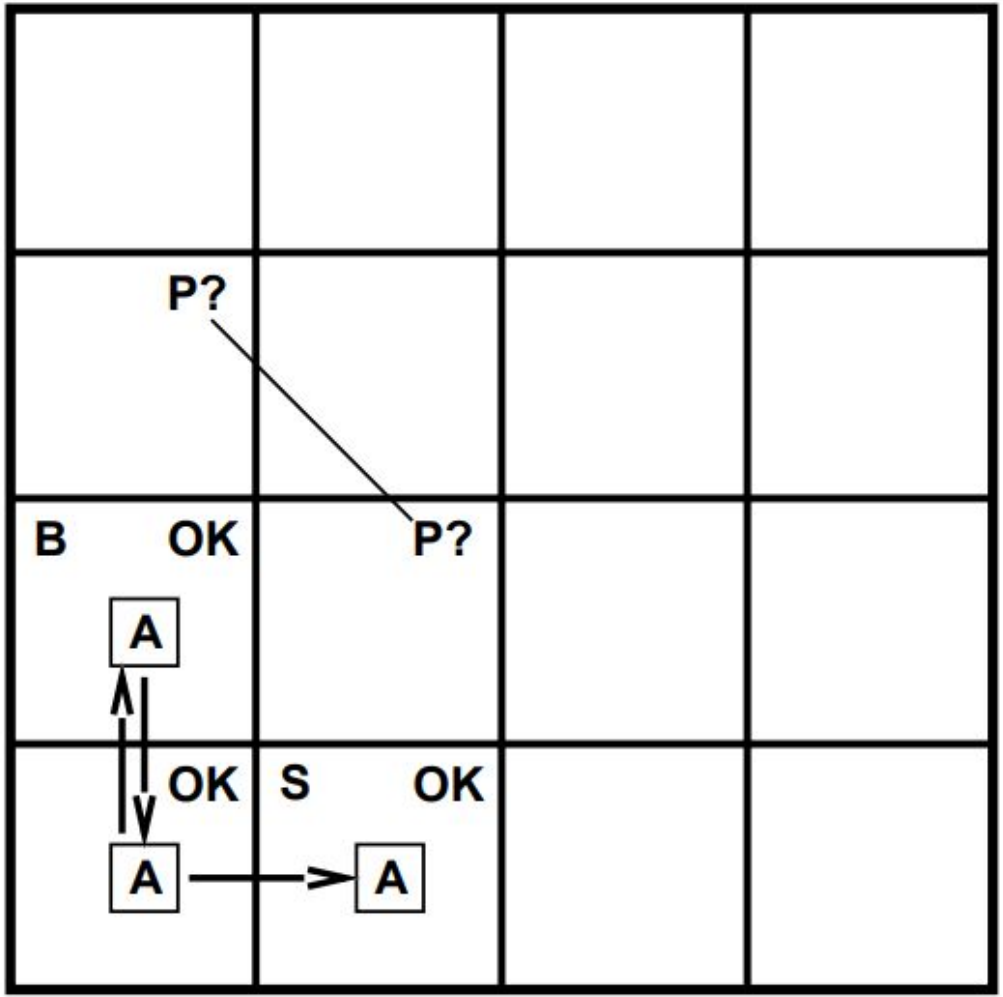
Exploring a wumpus world

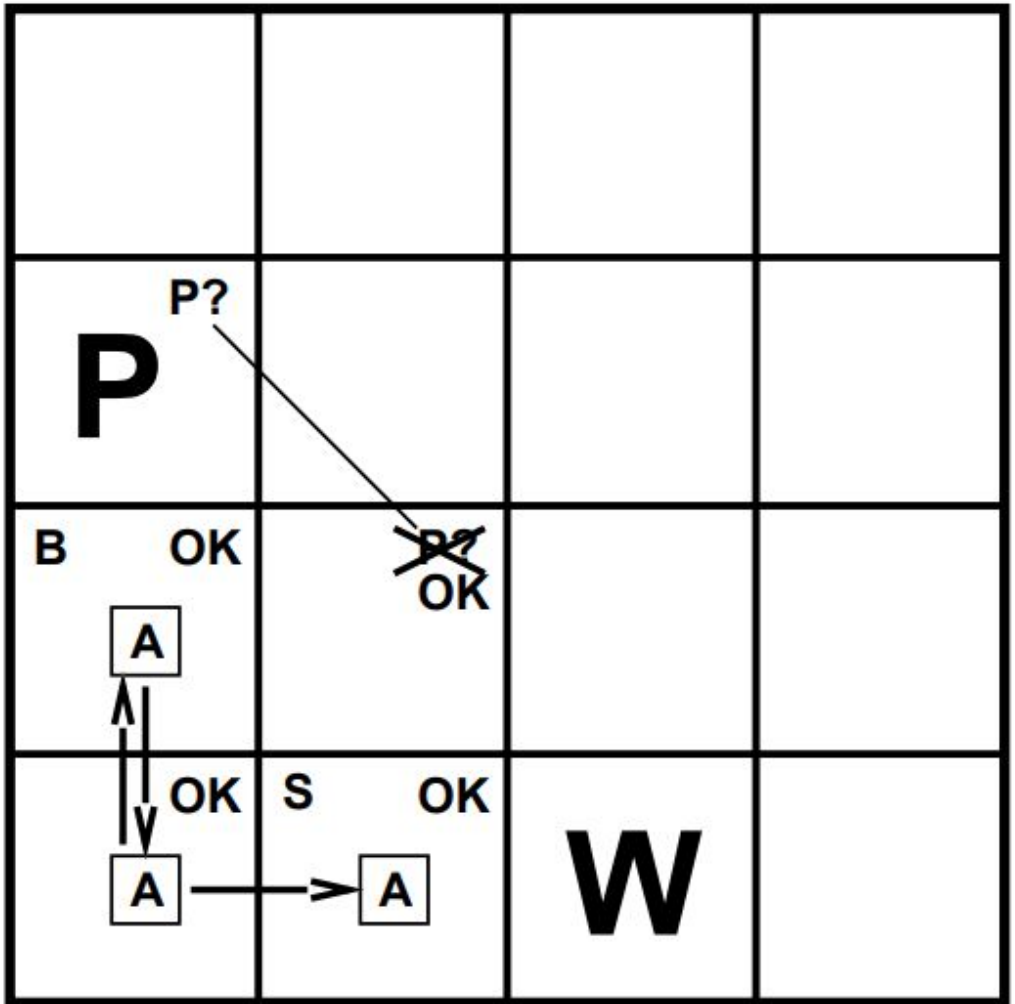


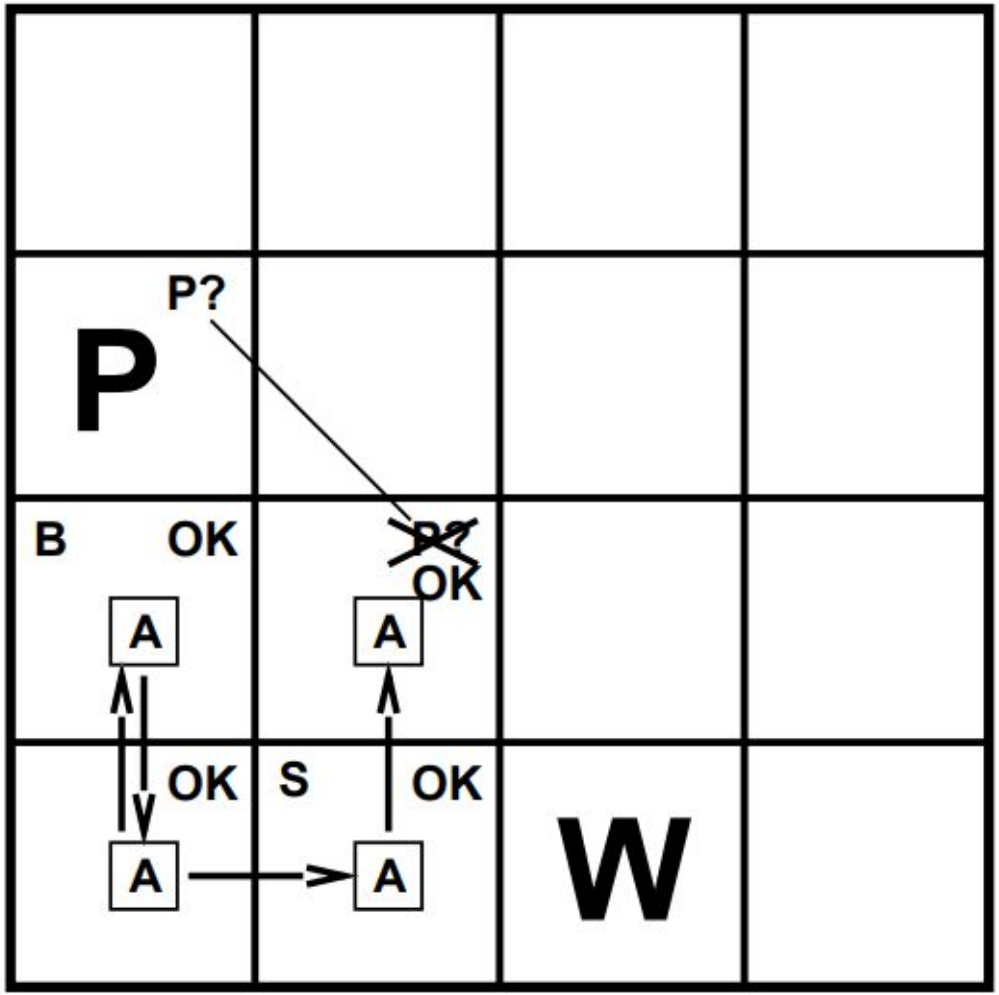
- Percept sequence yields nontrivial inferences (safe squares, possible pits/wumpus).
- Absence of a percept can be informative.

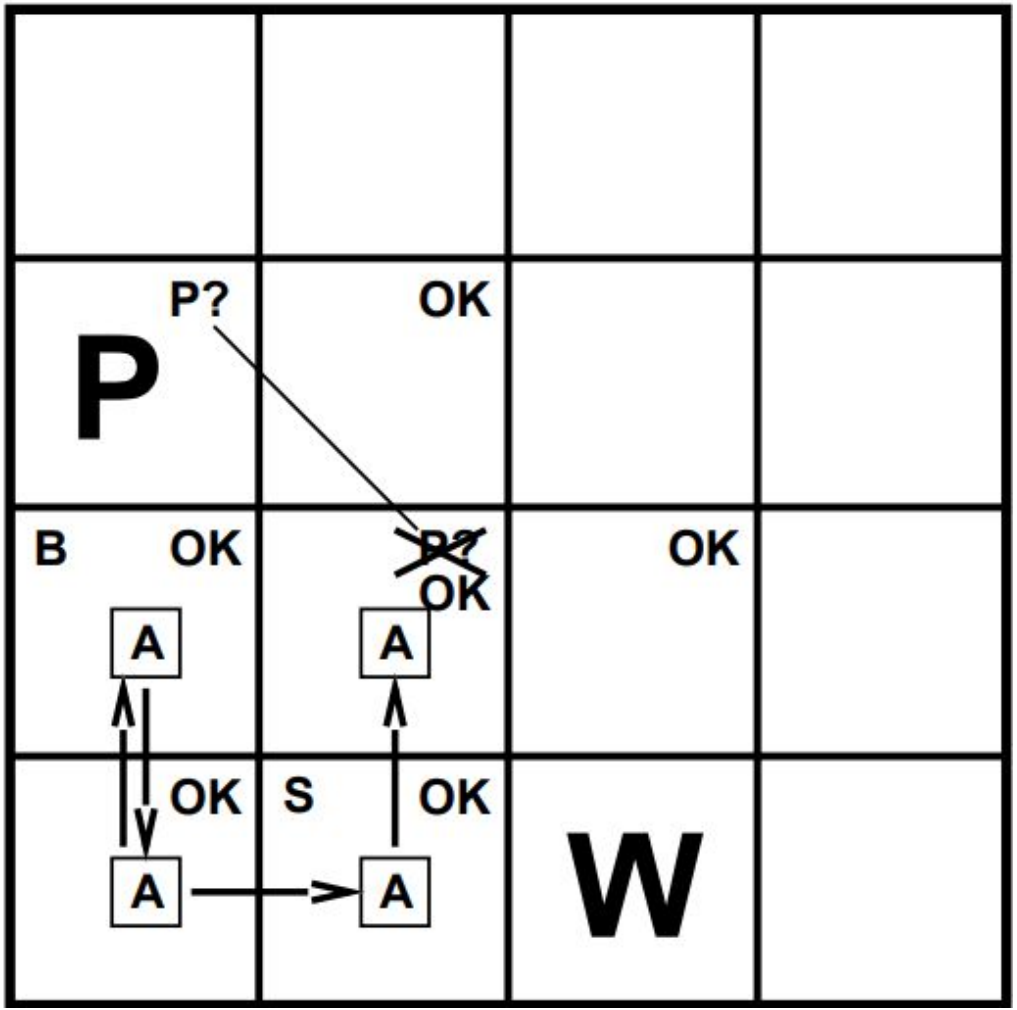

B
OK
A
OK
OK
A

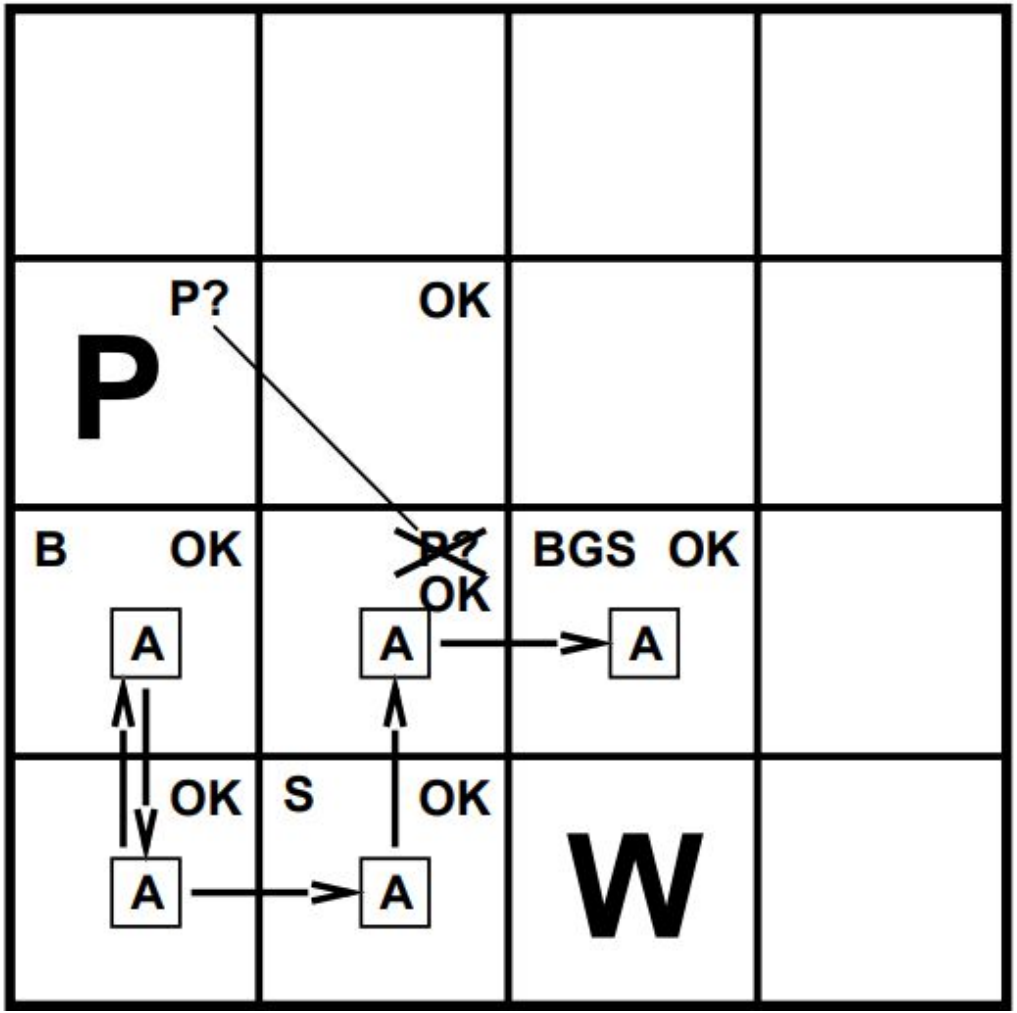

P?
B
OK
P?
A
OK
OK
A


P?
B
OK
P?
A
OK
S
OK
A
A

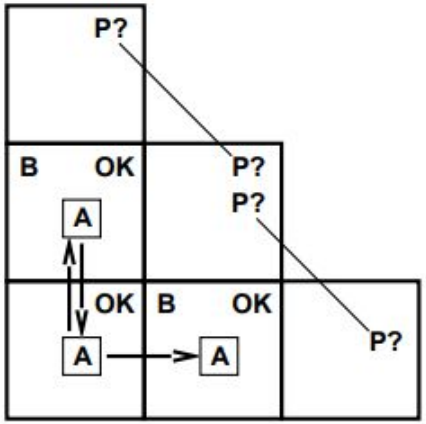

P?
P
B
OK
P?
OK
A
OK
S
OK
A
A
W


P?
P
B
OK
P?
OK
A
A
OK
S
OK
A
A
W


P?
OK
P
B
OK
P?
OK
OK
A
A
OK
S
OK
A
A
W

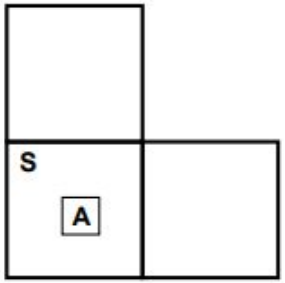

P?
OK
P
B
OK
P?
OK
BGS
OK
A
A
A
OK
S
OK
A
A
W

Other tight spots



Breeze in (1,2) and (2,1)
⇒ no safe actions

Assuming pits uniformly distributed,
(2,2) is most likely to have a pit



Smell in (1,1)
⇒ cannot move
Can use a strategy of coercion:
shoot straight ahead
wumpus was there ⇒ dead ⇒ safe
wumpus wasn't there ⇒ safe

Logic in general

Logics are formal languages for representing information
such that conclusions can be drawn

Syntax defines the sentences in the language

Semantics define the "meaning" of sentences;
i.e., define truth of a sentence in a world

E.g., the language of arithmetic

$x+2 \geq y$ is a sentence; $x2+y >$ is not a sentence

$x+2 \geq y$ is true iff the number $x+2$ is no less than the number y

$x+2 \geq y$ is true in a world where $x=7,\ y=1$
$x+2 \geq y$ is false in a world where $x=0,\ y=6$

Types of logic

Logics are characterized by what they commit to as "primitives"

Ontological commitment: what exists—facts? objects? time? beliefs?

Epistemological commitment: what states of knowledge?

Language	Ontological Commitment	Epistemological Commitment
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief 0...1
Fuzzy logic	degree of truth	degree of belief 0...1

Entailment

$KB \models \alpha$

Knowledge base KB entails sentence α
if and only if
α is true in all worlds where KB is true

E.g., the KB containing "the Giants won" and "the Reds won"
entails "Either the Giants won or the Reds won"

Models

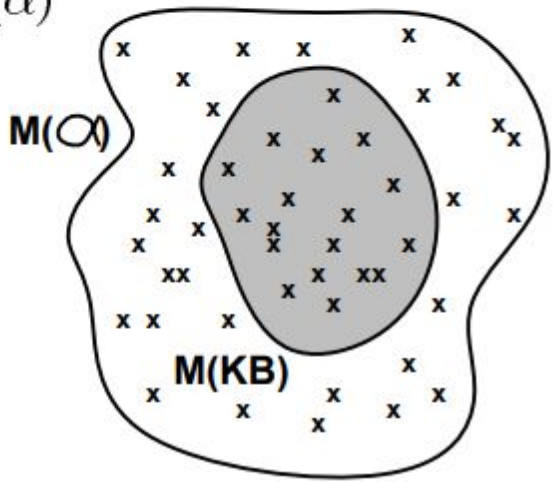
Logicians typically think in terms of models, which are formally structured worlds with respect to which truth can be evaluated

We say m is a model of a sentence α if α is true in m

$M(\alpha)$ is the set of all models of α

Then $KB \models \alpha$ if and only if $M(KB) \subseteq M(\alpha)$

E.g. $KB =$ Giants won and Reds won
$\alpha =$ Giants won



Inference

$KB \vdash_i \alpha =$ sentence α can be derived from KB by procedure i

Soundness: i is sound if
whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if
whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

Preview: we will define a logic (first-order logic) which is expressive enough to say almost anything of interest, and for which there exists a sound and complete inference procedure.

That is, the procedure will answer any question whose answer follows from what is known by the KB.

Propositional logic: Syntax

Propositional logic is the simplest logic—illustrates basic ideas

The proposition symbols P_1, P_2 etc are sentences

If S is a sentence, $\neg S$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \wedge S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \vee S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \Rightarrow S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \Leftrightarrow S_2$ is a sentence

Propositional logic: Semantics

Each model specifies true/false for each proposition symbol

E.g.

A	B	C
$True$	$True$	$False$

Rules for evaluating truth with respect to a model m:

$\neg S$	is true iff	S	is false				
$S_1 \wedge S_2$	is true iff	S_1	is true and	S_2	is true		
$S_1 \vee S_2$	is true iff	S_1	is true or	S_2	is true		
$S_1 \Rightarrow S_2$	is true iff	S_1	is false or	S_2	is true		
i.e.,	is false iff	S_1	is true and	S_2	is false		
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$	is true and	$S_2 \Rightarrow S_1$	is true		

Propositional inference: Enumeration method

Let $\alpha = A \vee B$ and $KB = (A \vee C) \wedge (B \vee \neg C)$

Is it the case that $KB \models \alpha$?
Check all possible models—α must be true wherever KB is true

A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
$False$	$False$	$False$				
$False$	$False$	$True$				
$False$	$True$	$False$				
$False$	$True$	$True$				
$True$	$False$	$False$				
$True$	$False$	$True$				
$True$	$True$	$False$				
$True$	$True$	$True$				

Propositional inference: Solution

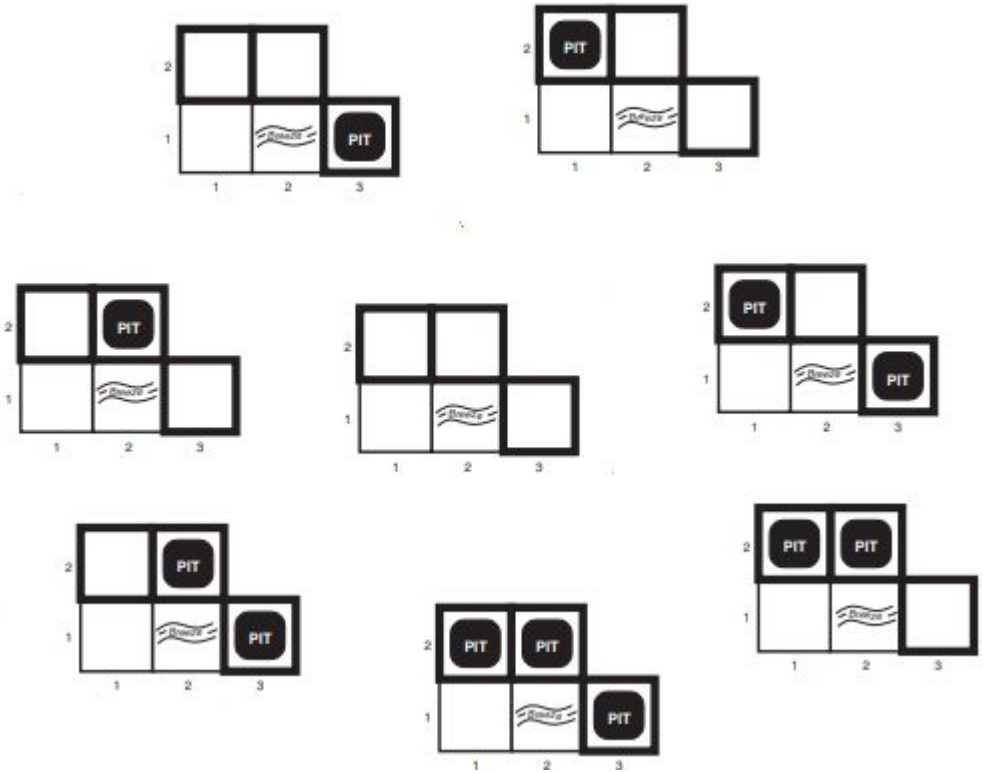
A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
$False$	$False$	$False$	$False$	$True$	$False$	$False$
$False$	$False$	$True$	$True$	$False$	$False$	$False$
$False$	$True$	$False$	$False$	$True$	$False$	$True$
$False$	$True$	$True$	$True$	$True$	$True$	$True$
$True$	$False$	$False$	$True$	$True$	$True$	$True$
$True$	$False$	$True$	$True$	$False$	$False$	$True$
$True$	$True$	$False$	$True$	$True$	$True$	$True$
$True$	$True$	$True$	$True$	$True$	$True$	$True$

Wumpus World: model-checking example

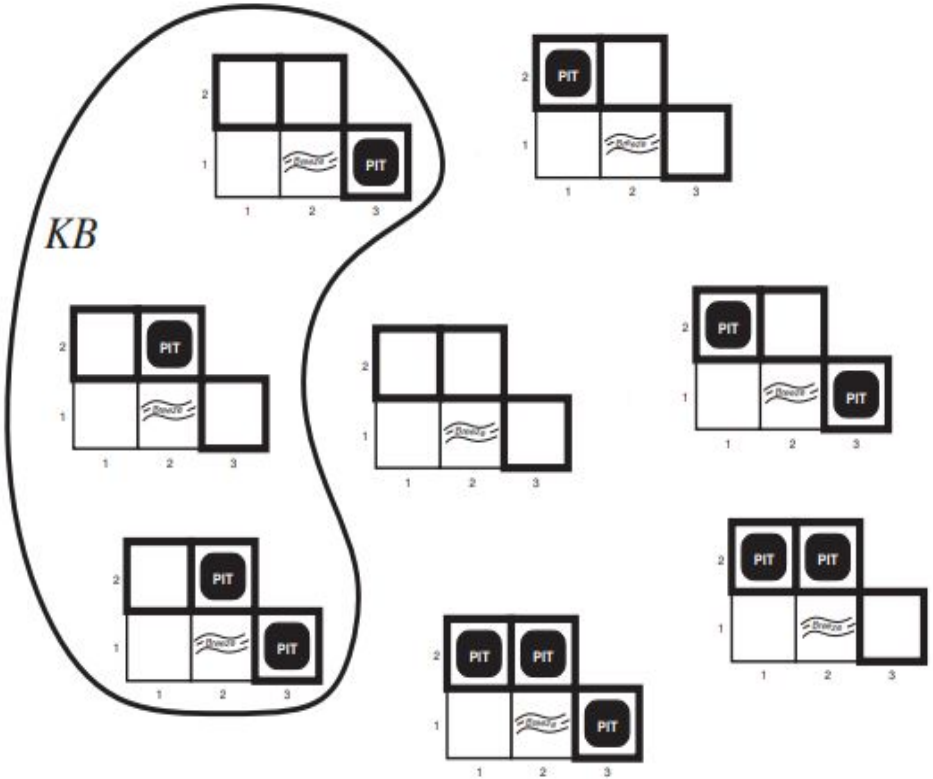
Setup (percepts + question)

- Percepts: **no breeze** in [1,1]; **breeze** in [2,1].
- KB={Percepts, Rules of the Wumpus world}
- Agent asks whether squares **[1,2], [2,2], [3,1]** contain pits.
- Each of the three squares may or may not have a pit → 2^3 =8 possible models.

Wumpus World: model-checking example



Wumpus World: model-checking example



KB constraints (from percepts + world rules)

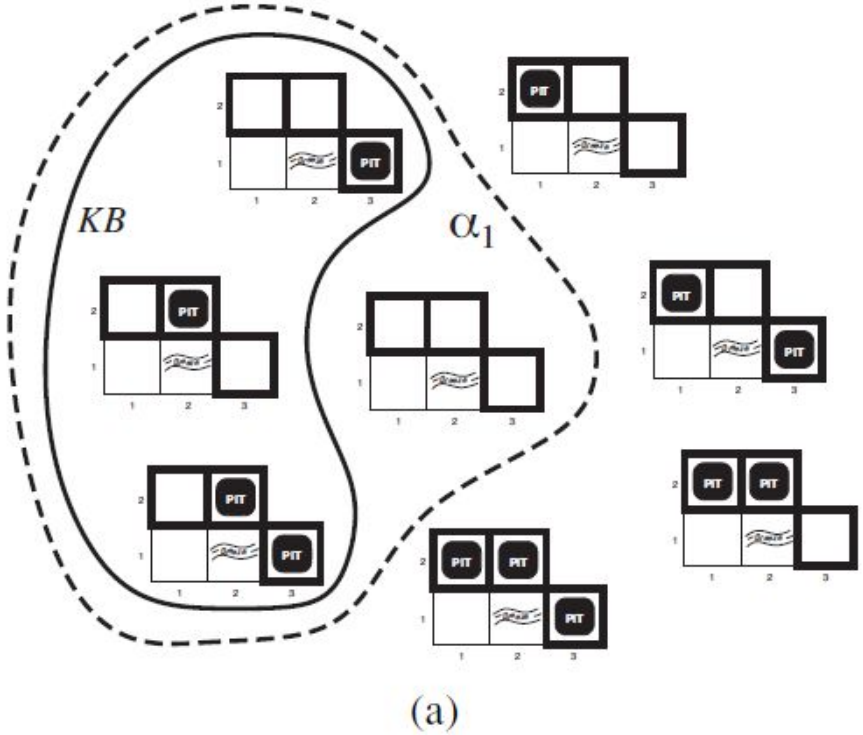
1. No breeze in [1,1] ⇒ **no pit in any neighbor of [1,1]** ⇒ P1,2=0 and P2,1=0, .

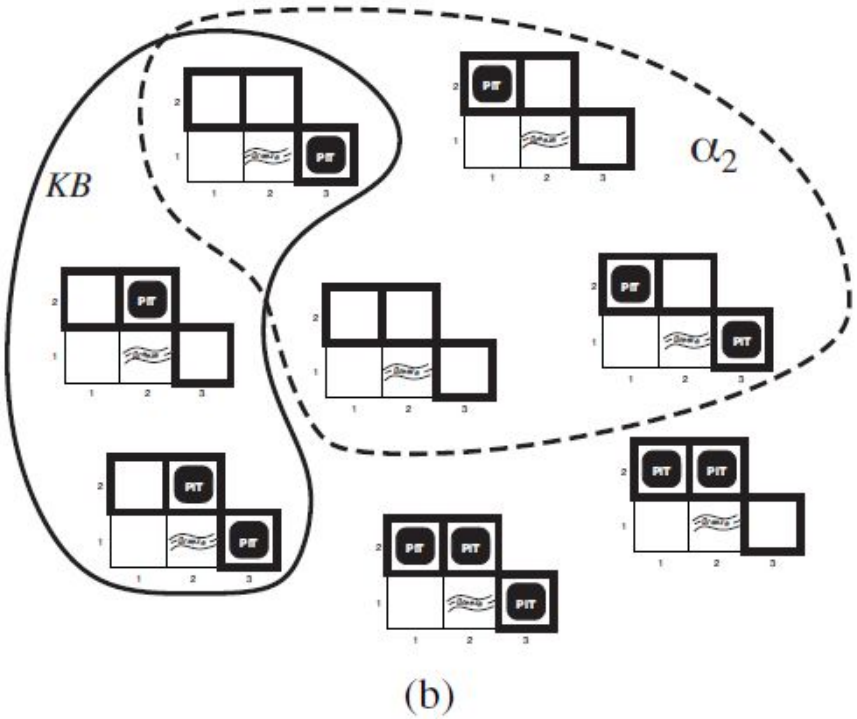
2. Breeze in [2,1] ⇒ **at least one neighbor of [2,1] has a pit**. Neighbors: [1,1], [2,2], [3,1]. Since [1,1] has no pit, this implies **P2,2 OR I**

Wumpus World: model-checking example

α1: “There is **no** pit in [1,2]” (i.e., p1,2=0)
α2: “There is **no** pit in [2,2]” (i.e., p2,2=0)

$M(KB) \subseteq M(\alpha)$





Check entailment by model-checking

- Is KB⊨α1?→ In every model where KB is true, α1 is true **Yes**: KB entails α1

- Is KB⊨α2? → Among KB-true models, some have p2,2=1 and some have p2,2=0. **No**: KB does **not** entail α2 (nor its negation).

Model Checking vs Theorem Proving (worked toy example)

Model checking: enumerate all assignments (models) to propositional symbols and check a sentence in each model. - **Theorem proving:** apply inference rules to derive the target sentence from KB without enumerating every model.

Logical Equivalence

Two sentences are logically equivalent iff they are true in exactly the same models.

commutativity of AND: α1= P⋀Q and α2= Q⋀P.

P ⋀ Q equivalent to Q ⋀ P.

P	Q	P AND Q	Q AND P
T	T	T	T
T	F	F	F
F	T	F	F
F	F	F	F

Logical Equivalence

P => Q is equivalent to ¬ P ∨ Q.

Why useful: Equivalences allow rewriting formulas (e.g., to CNF) without changing meaning.

P	Q	P=>Q	¬ P ∨ Q
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Validity (Tautology)

A sentence is valid if it is true in all models (a tautology).
Valid sentence ≡ True

P	¬ P	P OR ¬ P
T	F	T
F	T	T

Deduction Theorem

For any sentences α and β, $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid.

$\alpha = P \wedge Q$ $\beta = P$

P	Q	P ∧ Q	(P ∧ Q) => P
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	T

Satisfiability and SAT

A sentence is satisfiable if there exists at least one model that makes it true.

F = (A OR B) AND (NOT A).

model: - If A = F, B = T → (A OR B) = T and NOT A = T, so F is True.
So F is satisfiable.

Formula: G = A AND NOT A.

No model can make both A and NOT A true simultaneously → unsatisfiable.

Validity vs. Unsatisfiability

A sentence α is **valid** if and only if ¬α is **unsatisfiable**.

Why: If α is true in **all** models, then there is **no** model where ¬α is true—making ¬α unsatisfiable.

Entailment via Unsatisfiability

α⊨β **iff** (α⋀¬β) is **unsatisfiable**.

- **Reasoning:** If α\alphaα entails β\betaβ, there is no model where α\alphaα is true **and** β\betaβ is false.

Inference Rules: Modus Ponens

Rule: $\alpha \Longrightarrow \beta$ and α, infer β

Example (Wumpus-style):

- **Sentence (Implication):** $(\text{WumpusAhead} \wedge \text{WumpusAlive}) \Longrightarrow \text{Shoot}$
- **Percept (Premise):** $\text{WumpusAhead} \wedge \text{WumpusAlive}$
- **Applying Modus Ponens yields:** Shoot

Given: $P \Longrightarrow (Q \vee R)$ and P

Infer: $Q \vee R$

Inference Rules: And-Elimination

From $\alpha \wedge \beta$, infer α (and similarly β).

Example:

From ItIsRaining $\wedge$ StreetsAreWet, infer StreetsAreWet

Biconditional Elimination.

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

De Morgan's Law Example

$$\neg(P \vee Q) \iff (\neg P \wedge \neg Q)$$

If you know $\neg(P \vee Q)$, you can infer $\neg P$ by **And-Elimination** after applying De Morgan's transformation.

Monotonicity

Property: Adding new sentences to KB cannot invalidate an already valid entailment.

Example: - Suppose KB entails NOT P_1,2 as shown earlier. - Add a new sentence beta: “There are exactly 8 pits in the world.” - Because propositional logic is monotonic, KB1 AND beta still entails NOT P_1,2 — the original proof still holds (the new fact cannot retroactively change the truth of previous premises).

Resolution Rule

Unit Resolution Inference Rule:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

- unit resolution rule takes a clause—a disjunction of literals—and a literal and produces a new clause.
- single literal can be viewed as a disjunction of UNIT CLAUSE one literal, also known as a unit clause.
- the resulting clause should contain FACTORING only one copy of each literal

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

Resolution Rule

Example:

$$\frac{P_{1,1} \vee P_{3,1}, \qquad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

CNF

Def: A CONJUNCTIVE sentence expressed as a conjunction of clauses is said to be in conjunctive normal form or NORMAL FORM (CNF)

Example: convert the following into CNF $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

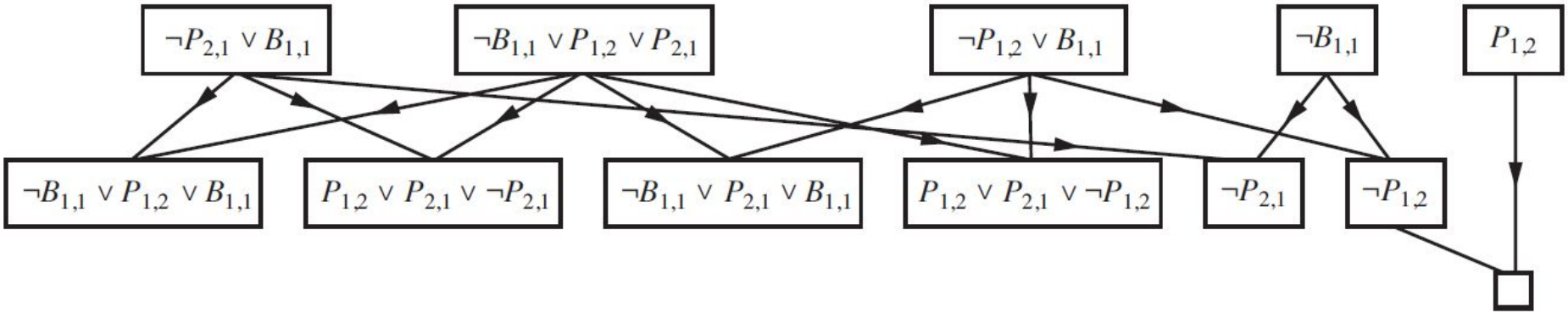
CNF $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

Resolution Algorithm

function PL-RESOLUTION(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  $clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
  $new \leftarrow \{\,\}$
  loop do
    for each pair of clauses C_i, C_j **in** $clauses$ **do**
      $resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)
      if $resolvents$ contains the empty clause **then return** *true*
      $new \leftarrow new \cup resolvents$
    if $new \subseteq clauses$ **then return** *false*
    $clauses \leftarrow clauses \cup new$

Example: Resolution Algo.



Summary: Resolution rule

- Clause form (CNF): a clause is a disjunction of literals (e.g. $P \vee \neg Q$. (Literal: either P or $\neg P$.)
- Resolution rule: from $(A \vee P)$ and $(B \vee \neg P)$ infer $(A \vee B)$
- Factoring: remove duplicate literals inside a clause (keeps clause space finite).

Definite clause

Definite clause: a clause with **exactly one positive literal**.

- E.g.: $\neg A \vee \neg B \vee C$ — exactly one positive literal C.
- Canonical form- $(A \wedge B) \Rightarrow C$
- Terminology:
  - **Body** (premise): conjunction of positive literals on left $(A \wedge B)$.
  - **Head** (conclusion): single positive literal on right C.
  - **Fact:** a definite clause with empty body (equivalently a single positive literal; e.g., A.

Horn clause

Horn clause: a clause with **at most one positive literal**.

- All definite clauses are Horn clauses (exactly one positive literal).
- Also includes clauses with **no** positive literal — these are **goal clauses**
- Important property: **Horn clauses are closed under resolution** — resolving two Horn clauses yields a Horn clause.

Closure under resolution

Suppose two Horn clauses are resolved on some literal:

- Clause1: ¬X ∨ A (at most one positive in total)
- Clause2: X ∨ ¬B
- Resolvent: A ∨ ¬B

Why Horn/Definite clauses matter

Implication form: Each definite clause is an implication with a conjunctive body and a single head — intuitive rules.
Efficient inference: Forward/backward chaining algorithms apply naturally and are easier to implement and follow.
Linear-time entailment: Deciding entailment for propositional Horn KBs can be done in time linear in the size of the KB.

Forward chaining algorithm (high level)

- Start with a set of known facts (heads with empty body).
- Repeatedly fire any rule whose body is satisfied by known facts, adding its head as a new fact.
- Continue until no new facts can be added (or query is found).

Reduction to SAT

Model Finding & SAT (Satisfiability)

- model finding (finding a satisfying assignment), SAT as NP-complete, and how to encode real problems into CNF for SAT solvers.
- Concrete worked examples: circuit equivalence, N-Queens, map-coloring (CSP→SAT), and model-based diagnosis.

Logical identities to remember

Use these equivalences when converting and simplifying before final CNF conversion:

- De Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B$, $\neg(A \vee B) \equiv \neg A \wedge \neg B$.
- Distributivity: $(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$
- Absorption: $A \vee (A \wedge B) \equiv A$

XOR simplification: $X \oplus Y \equiv (X \vee Y) \wedge (\neg X \vee \neg Y)$— this is already in CNF if X and Y are atoms; if X or Y are compound, substitute then simplify.

Example 1

Circuits:

- C1: output $o = A \wedge B$.
- C2: output $o' = \neg(A \wedge B)$ i.e. $o' = \neg A \vee \neg B$

Goal: testing satisfiability of $(A \wedge B) \oplus (\neg A \vee \neg B)$ (i.e. if XOR is true in some world mean then circuits are different)

Step 1 — Express XOR as CNF skeleton:

$(X \oplus Y) \equiv (X \vee Y) \wedge (\neg X \vee \neg Y)$ with $X = A \wedge B$, $Y = \neg A \vee \neg B$

Step 2 — Substitute and simplify:

1. First clause: $(A \wedge B) \vee (\neg A \vee \neg B)$
   - By associativity/commutativity, this simplifies: $(A \wedge B) \vee \neg A \vee \neg B$
   - Use absorption: $\neg A \vee (A \wedge B) \equiv \neg A \vee B$— more simply, the whole clause is a tautology
2. Second clause: $\neg(A \wedge B) \vee \neg(\neg A \vee \neg B)$

   Simplify: $(\neg A \vee \neg B) \vee (A \wedge B)$ — which is the same as the first clause (commutative).

Final CNF after simplification: the XOR is a tautology (always true) because o′o'o′ is exactly the negation of ooo; so the formula is trivially SAT for any input.

Solver result: SAT — any input is a witness; solver returns e.g. A=0,B=0 as a counterexample (since outputs differ).

Example 2

C1: $o = A \wedge B$
C2: $o' = A \vee B$
First clause: $A \vee B$
Second Clause: $\neg A \vee \neg B$

Final CNF: $(A \vee B) \wedge (\neg A \vee \neg B)$

SAT models: (A=1,B=0) and (A=0,B=1).

n-Queen

Problem statement

Place n queens on an n×n chessboard so that no two queens attack each other. We model a Boolean variable = true iff there is a queen at row i, column j.

A solution is an assignment to all satisfying all constraints below.

n-Queen: Constraints (logical form)

1. **Exactly one queen per row i:**
   - At least one: $\bigvee_{j=1}^{n} Q_{i,j}$.
   - At most one: for all $j < k$, $\neg Q_{i,j} \vee \neg Q_{i,k}$.
2. **At most one queen per column j:** for all pairs of rows $i < i'$, $\neg Q_{i,j} \vee \neg Q_{i',j}$
   .
3. **At most one queen per diagonal:** for every pair of distinct cells (i, j) and (i', j') with $|i - i'| = |j - j'|$, add $\neg Q_{i,j} \vee \neg Q_{i',j'}$.

(These are all clauses or pairs of clauses and therefore already in CNF when written as above.)

Graph Coloring

Input: an undirected graph $G = (V, E)$ and an integer k (number of colors).

Task (decision): does there exist a function $c : V \to \{1, \ldots, k\}$ s.t. for every edge $(u, v) \in E$, $c(u) \neq c(v)$?

Decision form for SAT: produce a CNF formula $\Phi(G, k)$ that is satisfiable iff the graph is k-colorable.

Graph Coloring

Introduce Boolean variables:

$X_{v,i}$ = true iff vertex $v \in V$ is assigned color $i \in \{1, \ldots, k\}$.

Number of variables = $|V| \times k$.

Graph Coloring

1. **Exactly one color per vertex v:**
   - At least one: $\bigvee_{i=1}^{k} X_{v,i}$.
   - At most one: for all $i < j$, add $\neg X_{v,i} \vee \neg X_{v,j}$.
2. **Edge (adjacency) constraint:** for every edge $(u, v) \in E$ and every color i, add $\neg X_{u,i} \vee \neg X_{v,i}$ (so two adjacent vertices cannot share color i).

These clauses together form the CNF $\Phi(G, k)$.

Example

Vertices: $V = \{v_1, v_2, v_3\}$, colors $\{R, G, B\}$ (map R→1,G→2,B→3).

Variables:

- $X_{v1,R} = x_1, X_{v1,G} = x_2, X_{v1,B} = x_3$
- $X_{v2,R} = x_4, X_{v2,G} = x_5, X_{v2,B} = x_6$
- $X_{v3,R} = x_7, X_{v3,G} = x_8, X_{v3,B} = x_9$

Total variables = 9.

Inference 4: DPLL (Enumeration of ***Partial*** Models)

[Davis, Putnam, Loveland & Logemann 1962]

Version 1

```
dpll_1(pa){
  if (pa makes F false) return false;
  if (pa makes F true) return true;
  choose P in F;
  if (dpll_1(pa ∪ {P=0})) return true;
  return dpll_1(pa ∪ {P=1});
}
```

Returns true if F is satisfiable, false otherwise

DPLL Version 1

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$

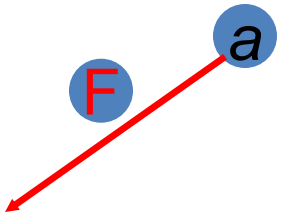
DPLL Version 1

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$



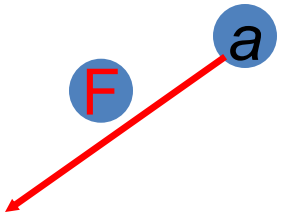
DPLL Version 1

$(F \vee b \vee c)$

$(F \vee \neg b)$

$(F \vee \neg c)$

$(T \vee c)$



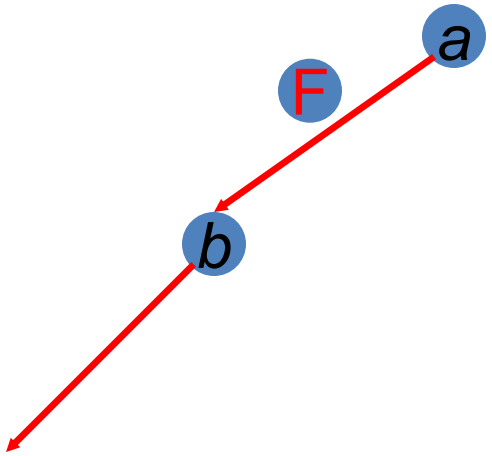
DPLL Version 1

$(F \vee F \vee c)$

$(F \vee T)$

$(F \vee \neg c)$

$(T \vee c)$



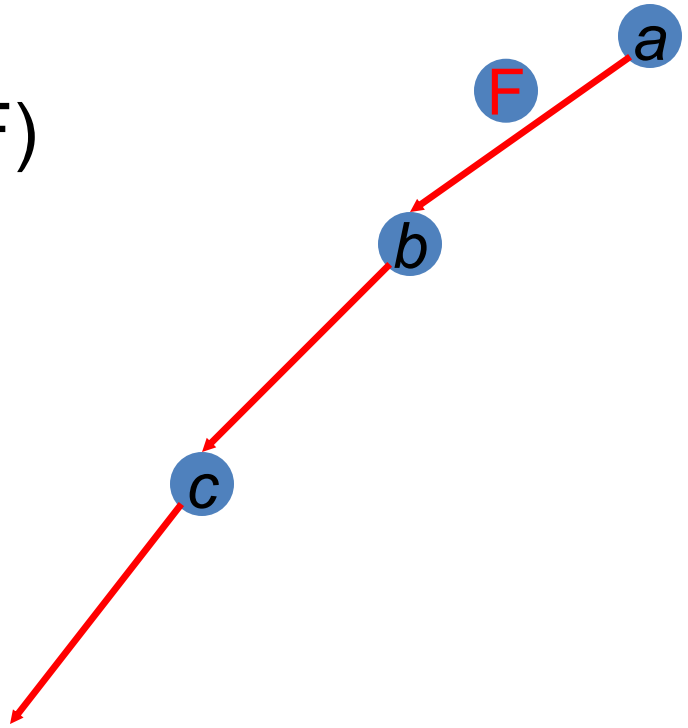
DPLL Version 1

$(F \vee F \vee F)$

$(F \vee T)$

$(F \vee T)$

$(T \vee F)$



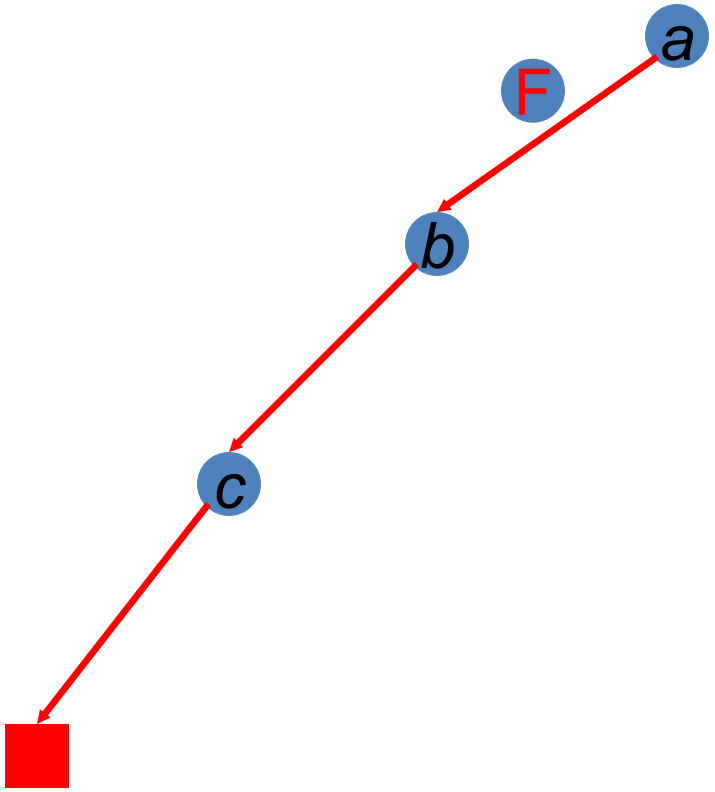
DPLL Version 1

F

T

T

T



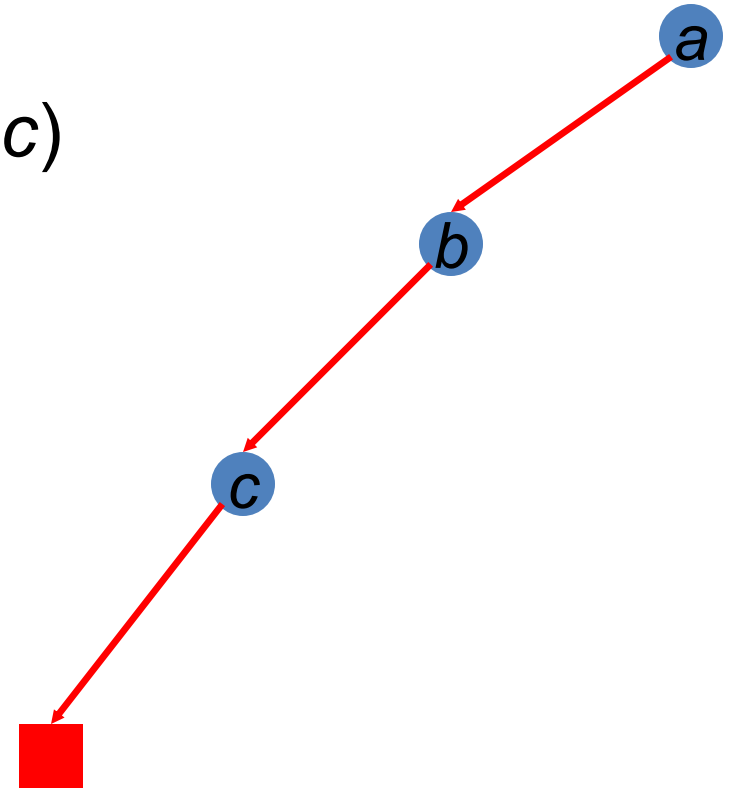
DPLL Version 1

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$



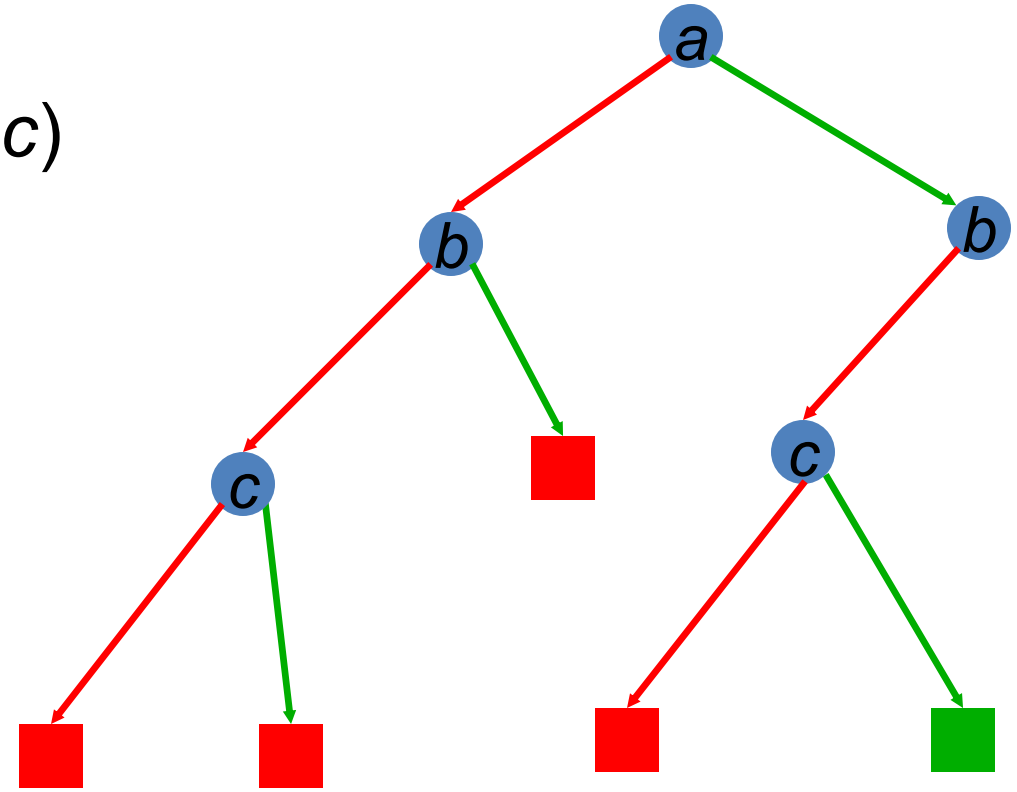
DPLL Version 1

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$



DPLL as Search

- Search Space?
- Algorithm?

Improving DPLL

If literal L_1 is true, then clause $(L_1 \vee L_2 \vee ...)$ is true

If clause C_1 is true, then $C_1 \wedge C_2 \wedge C_3 \wedge ...$ has the same value as $C_2 \wedge C_3 \wedge ...$

Therefore: Okay to delete clauses containing true literals!

If literal L_1 is false, then clause $(L_1 \vee L_2 \vee L_3 \vee ...)$ has the same value as $(L_2 \vee L_3 \vee ...)$

Therefore: Okay to shorten clauses containing false literals!

If literal L_1 is false, then clause (L_1) is false

Therefore: the empty clause means false!

DPLL version 2

```
dpll_2(F, literal){
  remove clauses containing literal
  if (F contains no clauses)return true;
  shorten clauses containing ¬literal
  if (F contains empty clause)
     return false;
  choose V in F;
  if (dpll_2(F, ¬V))return true;
  return dpll_2(F, V);
}
```

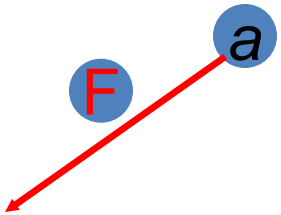
DPLL Version 2

$(F \vee b \vee c)$

$(F \vee \neg b)$

$(F \vee \neg c)$

$(T \vee c)$

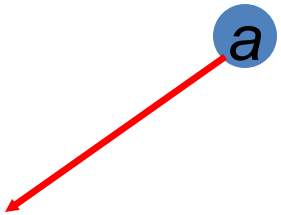


DPLL Version 2

$(b \vee c)$

$(\neg b)$

$(\neg c)$

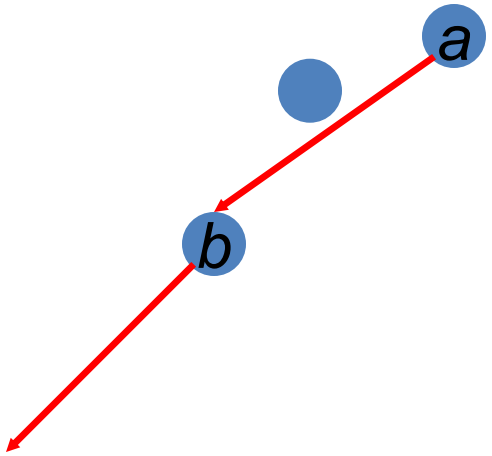


DPLL Version 2

(F $\vee$ *c*)

(T)

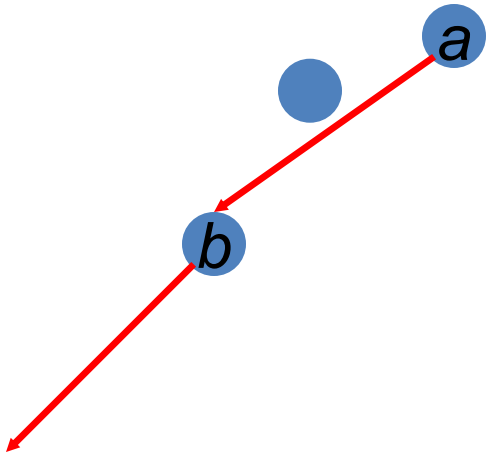
(¬*c*)



DPLL Version 2

(*c*)

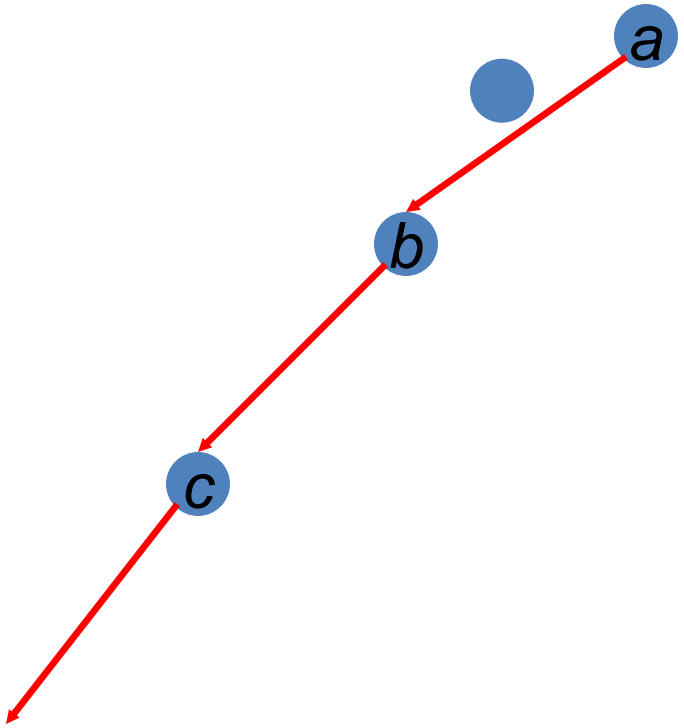
(¬*c*)



DPLL Version 2

(F)

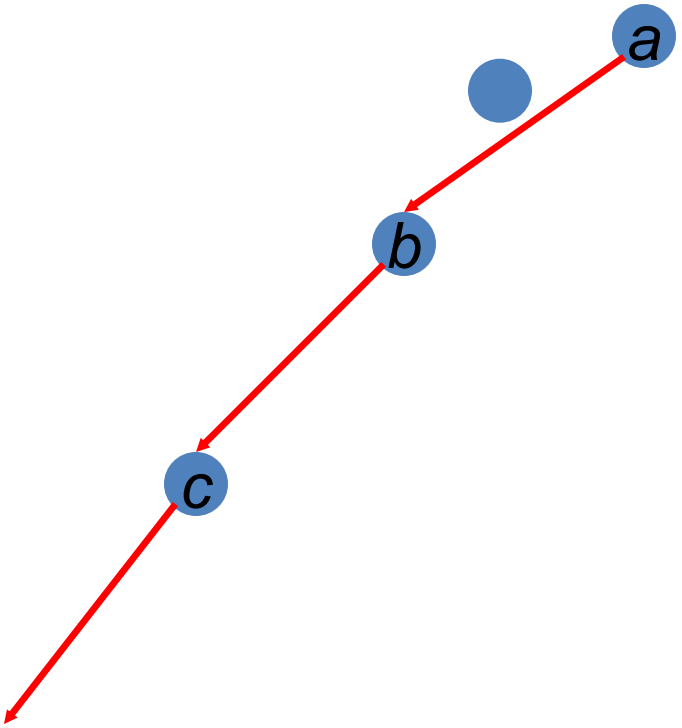
(T)



DPLL Version 2

Empty clause!

()



Structure in Clauses

- Unit Literals (unit propagation)

  A literal that appears in a singleton clause

  {{¬b c}{¬c}{a ¬b e}{d b}{e a ¬c}}

  Might as well set it true! And simplify

  {{¬b} {a ¬b e}{d b}}

  {{d}}

- Pure Literals
  - A symbol that always appears with same sign
  - {{a ¬b c}{¬c d ¬e}{¬a ¬b e}{d b}{e a ¬c}}

  Might as well set it true! And simplify

  {{a ¬b c} {¬a ¬b e} {e a ¬c}}

In Other Words

Formula $(L) \wedge C_2 \wedge C_3 \wedge \ldots$ is only true when literal L is true

Therefore: Branch immediately on unit literals!

May view this as adding constraint propagation techniques into play

In Other Words

Formula $(L) \wedge C_2 \wedge C_3 \wedge \ldots$ is only true when literal L is true

Therefore: Branch immediately on unit literals!

If literal L does not appear negated in formula F, then setting L true preserves satisfiability of F

Therefore: Branch immediately on pure literals!

May view this as adding constraint propagation techniques into play

DPLL (previous version)

Davis – Putnam – Loveland – Logemann

```
dpll(F, literal){
  remove clauses containing literal
  if (F contains no clauses) return true;
  shorten clauses containing ¬literal
  if (F contains empty clause)
     return false;


  choose V in F;
  if (dpll(F, ¬V))return true;
  return dpll(F, V);
}
```

DPLL (for real!)

Davis – Putnam – Loveland – Logemann

```
dpll(F, literal){
  remove clauses containing literal
  if (F contains no clauses) return true;
  shorten clauses containing ¬literal
  if (F contains empty clause)
     return false;
  if (F contains a unit or pure L)
     return dpll(F, L);
  choose V in F;
  if (dpll(F, ¬V))return true;
  return dpll(F, V);
}
```

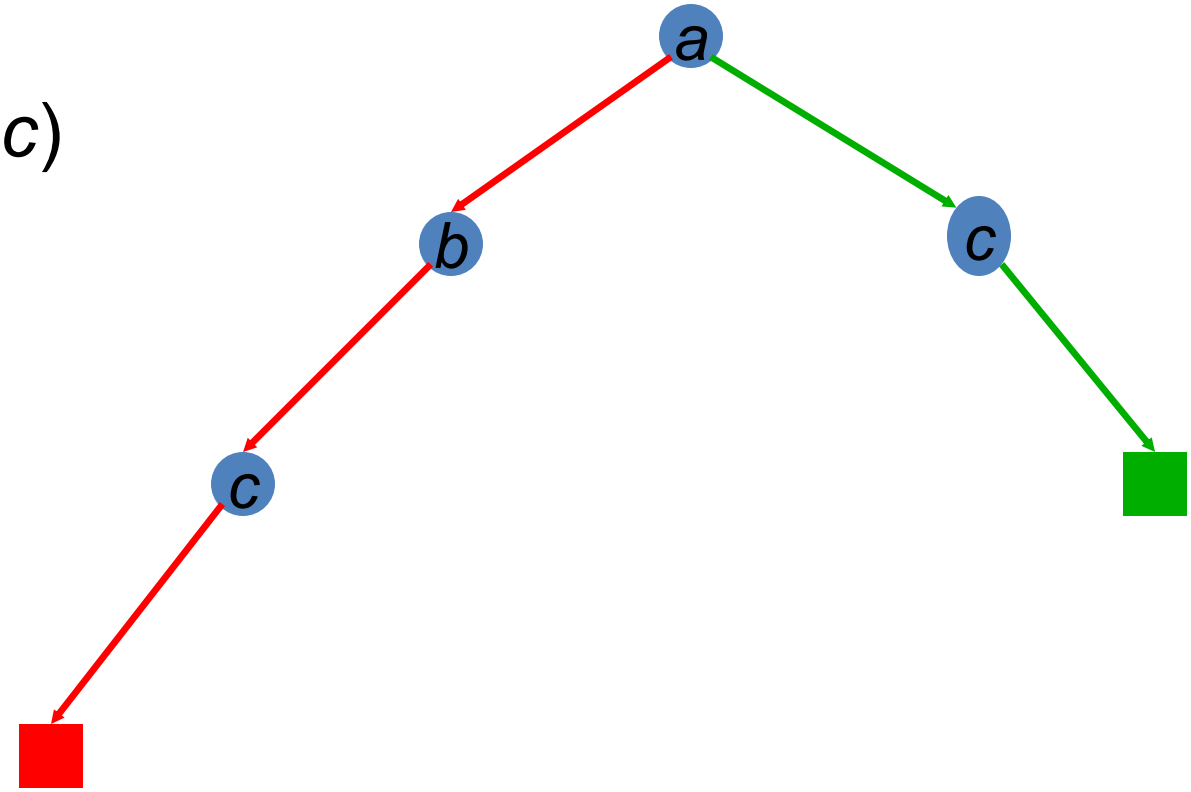
DPLL (for real)

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$



DPLL (for real!)

Davis – Putnam – Loveland – Logemann

```
dpll(F, literal){
   remove clauses containing literal
   if (F contains no clauses) return true;
   shorten clauses containing ¬literal
   if (F contains empty clause)
       return false;
   if (F contains a unit or pure L)
       return dpll(F, L);
   choose V in F;
   if (dpll(F, ¬V))return true;
   return dpll(F, V);
}
```

Where could we use a heuristic to further improve performance?

Heuristic Search in DPLL

- Heuristics are used in DPLL to select a (non-unit, non-pure) proposition for branching

- Idea: identify a most constrained variable
  - Likely to create many unit clauses
- MOM's heuristic:
  - Most occurrences in clauses of minimum length